A PROJECT REPORT ON

BloodConnect: Real-Time Blood Donation Network & Compatibility Matching System

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Computer Applications (Honours)

Course: Full Stack Web Development Project using ASP.NET Core (Course Code: DSC-M-BCA-471P)

PRESENTED BY

Rajput Prince Singh (Roll No: 83)

BCA (Hons.)

SUBMITTED TO

Gujarat University

Ahmedabad, Gujarat

UNDER THE GUIDANCE OF

Prof. Sejal Shah

Assistant Professor

Lokmanya College of Computer Applications

Ahmedabad, Gujarat, India

Academic Year 2026–27

LOKMANYA COLLEGE OF COMPUTER APPLICATIONS

Affiliated to Gujarat University · Ahmedabad, Gujarat

CERTIFICATE

This is to certify that the project report entitled **"BloodConnect: Real-Time Blood Donation Network & Compatibility Matching System"** is a bonafide record of the work carried out by **Rajput Prince Singh** and submitted to **Gujarat University** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Computer Applications (Honors)** for the course **Full Stack Web Development Project using ASP.NET Core (DSC-M-BCA-471P)** during the academic year **2026–27**.

To the best of our knowledge, this work is original, was carried out at **Lokmanya College of Computer Applications, Ahmedabad**, under the guidance of **Prof. Sejal Shah**, and has not been submitted in part or in full to any other university or institution for the award of any degree or diploma.

Prof. Sejal Shah	**Dr. Prakash Khuman**
Project Guide	Director
Lokmanya College of Computer Applications	Lokmanya College of Computer Applications

Place: Ahmedabad

Date: ____ / ____ / 2026

INDEX / TABLE OF CONTENTS

Sr. No.	Description	Page No.
1.	Title Page	Page i
2.	College Certificate	Page ii
3.	Index (Table of Contents)	Page iii
4.	Acknowledgement	Page iv
5.	Project Overview & Architecture Scope	Page 5
6.	PHASE 1: MVC Setup, Base Layout & Authentication	Page 6
7.	PHASE 2: Forms, Data Validation & Blood Compatibility Logic	Page 7
8.	PHASE 3: Database Integration, Pledge Workflow & CRUD	Page 9
9.	PHASE 4: REST API, Cookie Authentication & Live Deployment	Page 11
10.	Testing Verification Matrix & Challenges	Page 13
11.	References & Bibliography	Page 14

Acknowledgement

Completing this project has been a remarkable learning experience in enterprise web development and application security, and I am grateful to all those who guided and supported me along the way.

First, I am deeply thankful to the management of **Lokmanya College of Computer Applications, Ahmedabad**, for providing state-of-the-art laboratory facilities, robust computing infrastructure, and a conducive academic environment.

I express my sincere gratitude to **Dr. Prakash Khuman**, Director, Lokmanya College of Computer Applications, for his constant encouragement and administrative support throughout the BCA (Honors) curriculum.

I am profoundly indebted to my project guide, **Prof. Sejal Shah**, for her expert mentorship, invaluable technical insights, and rigorous code reviews during every stage of the **BloodConnect** development lifecycle.

I am also thankful to **Gujarat University** for incorporating the advanced course **Full Stack Web Development Project using ASP.NET Core (DSC-M-BCA-471P)** into our academic structure, bridging theoretical software engineering with secure production-grade implementation.

Finally, I extend my heartfelt appreciation to my family and peers for their constant motivation and patience.

Rajput Prince Singh
BCA (Hons.) — Roll No: 83
Lokmanya College of Computer Applications

5. Project Overview & Architecture Scope

EXECUTIVE SUMMARY & SYSTEM SPECIFICATION

Project Name: BloodConnect: Real-Time Blood Donation Network & Compatibility Matching System

Domain: Emergency Healthcare Network / Blood Compatibility Matching / Full Stack Web Development

Tech Stack: ASP.NET Core 10 MVC, EF Core 10, SQL Server, ASP.NET Core Identity, SignalR WebSockets, IMemoryCache, Bootstrap 5

Key Security Features: Cookie Auth with OnValidatePrincipal Ban Eviction, Ownership-Scoped IDOR Guards (403/404), RowVersion Concurrency, Global CSRF.

Live Deployment: Hosted on MonsterASP via HTTPS (https://bloodconnector.runasp.net).

In medical emergencies, finding a compatible blood donor quickly is a life-or-death situation. Traditional methods such as phone calls, WhatsApp groups, and social media posts are slow and unreliable. **BloodConnect** is engineered as a centralized, real-time blood donation management platform that connects verified donors and patients in seconds with built-in red blood cell (RBC) compatibility matching, a two-way pledge workflow, and live SignalR WebSocket push notifications.

5.1 Core Architectural Principles

- **Strict MVC Separation:** Controllers remain thin and orchestrate workflows while medical domain logic (BloodCompatibility.cs), hybrid caching (MessageCache.cs), and push notifications (NotificationService.cs) are cleanly isolated.
- **Pledge-Based Donation Workflow:** Replaces passive contact sharing with an accountable state machine (Pledged -> Accepted -> Completed -> Fulfilled) enforced by a unique (RequestId, DonorId) index.
- **DTO Separation & Privacy:** API endpoints use DonorDto and BloodRequestDto models. Donor phone numbers are hidden in public search results and revealed only after a request owner accepts a pledge.
- **IDOR & Role Hierarchy Protection:** Enforces SuperAdmin > Admin > User role hierarchy with immutable SuperAdmin protection, self-ban prevention, and server-side ownership checks on every edit/delete.

6. PHASE 1: MVC Setup, Base Layout & Authentication

PHASE 1 ALIGNMENT — ARCHITECTURE & SECURITY BASELINE

Objective: Configure ASP.NET Core 10 MVC architecture and establish secure cookie authentication with real-time ban eviction.

Key Deliverables: MVC project skeleton targeting net10.0, ASP.NET Core Identity (AppUser), DbSeeder role hierarchy, responsive Bootstrap 5 layout, and authorized dashboard.

Phase 1 establishes the foundation of BloodConnect. The project was initialized targeting **.NET 10 (net10.0)** with Entity Framework Core 10 and ASP.NET Core Identity. Cookie authentication (*Identity.Application*) was configured with **HttpOnly = true**, **SameSite = Lax**, a 7-day sliding expiration, 5-attempt lockout protection, and a custom **OnValidatePrincipal** cookie event that immediately signs out banned users.

Architecture Implementation — Program.cs — Authentication & Cookie Security Configuration

```
builder.Services.AddIdentity<AppUser, IdentityRole>(options => {
    options.Password.RequiredLength = 6;
    options.Password.RequireDigit = true;
    options.Lockout.MaxFailedAccessAttempts = 5;
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5);
}).AddEntityFrameworkStores<ApplicationDbContext>();

builder.Services.ConfigureApplicationCookie(options => {
    options.Cookie.Name = "Identity.Application";
    options.Cookie.HttpOnly = true;
    options.Cookie.SameSite = SameSiteMode.Lax;
    options.ExpireTimeSpan = TimeSpan.FromDays(7);
    options.SlidingExpiration = true;
    options.Events.OnValidatePrincipal = async context => {
        var userManager = context.HttpContext.RequestServices
            .GetRequiredService<UserManager<AppUser>>();
        var user = await userManager.GetUserAsync(context.Principal!);
        if (user?.IsBanned == true) {
            context.RejectPrincipal();
            await context.HttpContext.SignOutAsync();
        }
    };
});
```

Verified Behavior / Output:

```
Configured Identity with 5-attempt lockout, HttpOnly cookie, and instant ban eviction.
```

The base layout (_**Layout.cshtml**) was built using Bootstrap 5 with a warm medical theme (Cream *#FDF6EC*, Burgundy *#7B1E28*, Gold *#D4A24C*), Inter typography, inline SVG icons, and a live notification bell. **HomeController** handles public landing pages, while **DashboardController** and **AdminController** are protected with *[Authorize]*.

7. PHASE 2: Forms, Data Validation & Blood Compatibility Logic

PHASE 2 ALIGNMENT — USER INTERACTION & VALIDATION

Objective: Implement secure interaction forms, server-side data validation, guest emergency requests, and red-cell blood compatibility matching.

Key Deliverables: Donor & BloodRequest models, age (18-65) & regex validators, 28 States + 8 UTs validation, MustBeTrueAttribute, and BloodCompatibility.cs.

Phase 2 focuses on user interaction and medical data integrity. The **Donor** model enforces age validation between **18 and 65 years** (*[Range(18, 65)]*), blood group regex validation (*^(A|B|AB|O)[+-]$*), and valid Indian State/UT selection (28 states + 8 UTs). Emergency blood requests can be posted by guests without login (nullable *UserId*), with global CSRF protection enabled via **[AutoValidateAntiforgeryToken]**.

7.1 Deterministic Red Blood Cell (RBC) Compatibility Engine

To eliminate incompatible matching during emergencies, real RBC transfusion rules (where **O-** is the universal donor and **AB+** is the universal receiver) are encoded in **BloodCompatibility.cs** using a C# switch expression for O(1) constant-time lookup:

Architecture Implementation — Models/BloodCompatibility.cs — O(1) Compatibility Matrix

```
public static class BloodCompatibility
{
    public static IReadOnlyList<string> CompatibleDonorsFor(string recipient) =>
        recipient switch {
            "O-"  => new[] { "O-" },
            "O+"  => new[] { "O-", "O+" },
            "A-"  => new[] { "O-", "A-" },
            "A+"  => new[] { "O-", "O+", "A-", "A+" },
            "B-"  => new[] { "O-", "B-" },
            "B+"  => new[] { "O-", "O+", "B-", "B+" },
            "AB-" => new[] { "O-", "A-", "B-", "AB-" },
            "AB+" => new[] { "O-", "O+", "A-", "A+", "B-", "B+", "AB-", "AB+" },
            _     => Array.Empty<string>()
        };
}
```

Verified Behavior / Output:

```
Deterministic RBC compatibility lookup applied across web search, API search, and pledges.
```

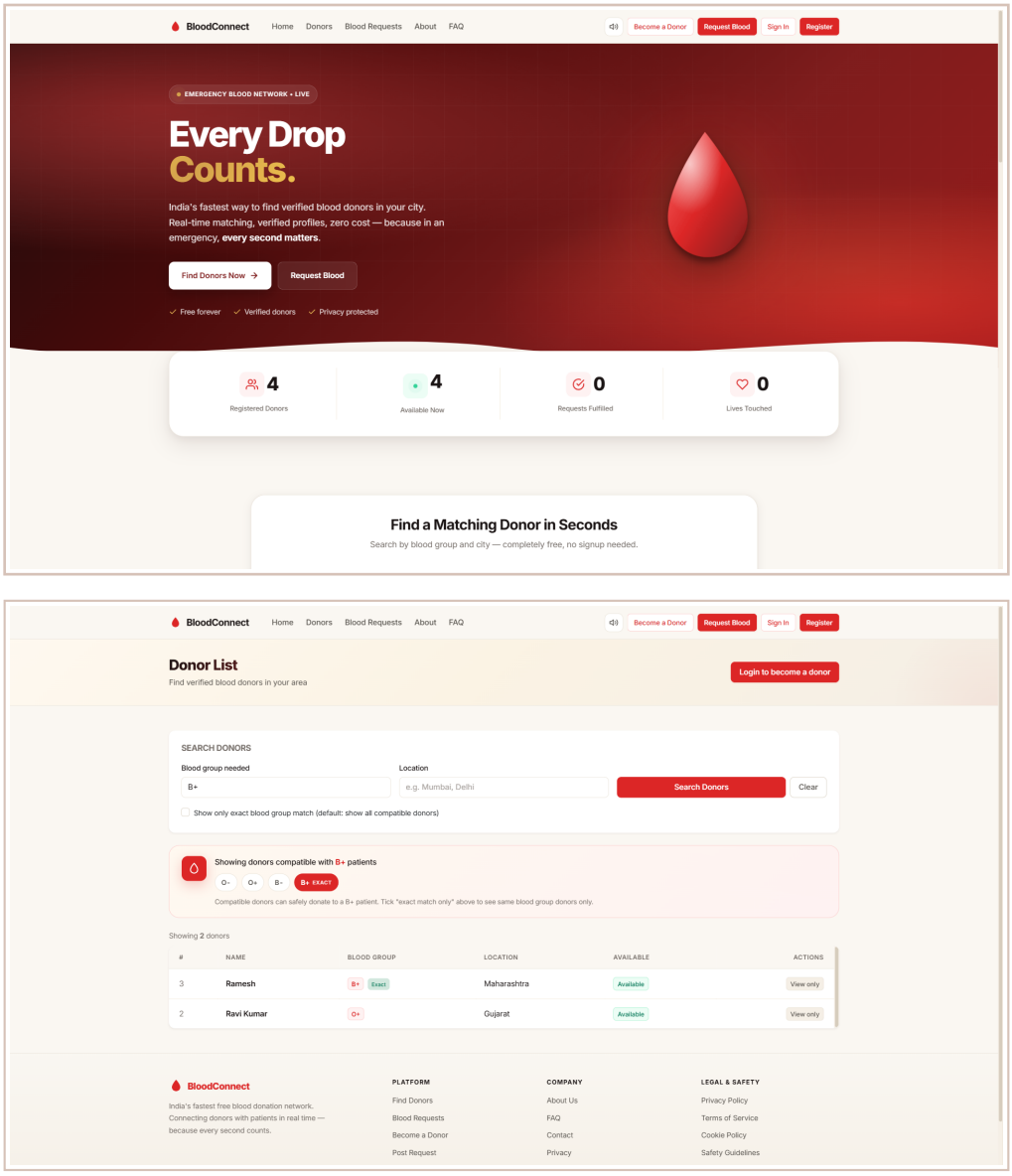


Figure 7.1: *BloodConnect Homepage Hero and Compatibility-Aware Donor Search filtering compatible donor groups (O-, O+, A-, A+) with privacy-masked phone numbers.*

8. PHASE 3: Database Integration, Pledge Workflow & CRUD

PHASE 3 ALIGNMENT — PERSISTENCE, PLEDGE WORKFLOW & CONCURRENCY

Objective: Establish EF Core 10 persistence, two-way DonationOffer pledge workflow, optimistic concurrency control, and cached admin inbox.

Key Deliverables: ApplicationDbContext (8 tables), DonationOffer state machine, [Timestamp] RowVersion concurrency token, and 60s IMemoryCache.

Phase 3 is the core persistence milestone of BloodConnect. Rather than publicly exposing donor phone numbers, donations follow a structured two-way **DonationOffer** state machine backed by EF Core 10:

- **Unique Index & Auto-Reject Transaction:** A unique compound index on (RequestId, DonorId) prevents duplicate pledges. When the owner accepts one offer, all competing offers are auto-rejected in a single transaction.
- **Optimistic Concurrency & 60s Cache:** RowVersion ([Timestamp] byte[]) on Donor prevents lost updates in concurrent edits, while MessageCache.cs caches unread admin message counts for 60 seconds.

Contact phone numbers remain strictly hidden in public search queries and are revealed only after mutual agreement when an offer transitions to the **Accepted** state.

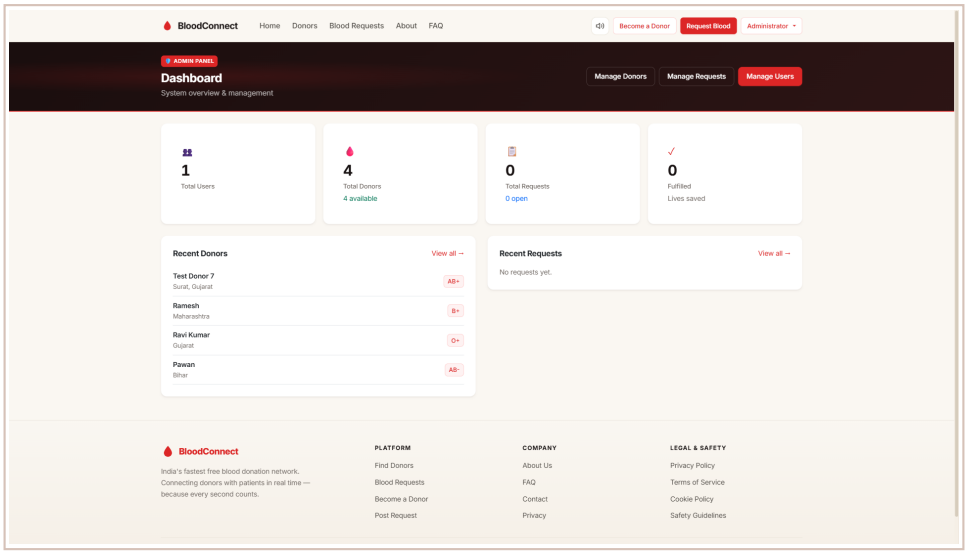


Figure 8.1: *Admin Dashboard & Database CRUD Management inspecting verified donors, active blood requests, role hierarchy controls, and cached unread message counts.*

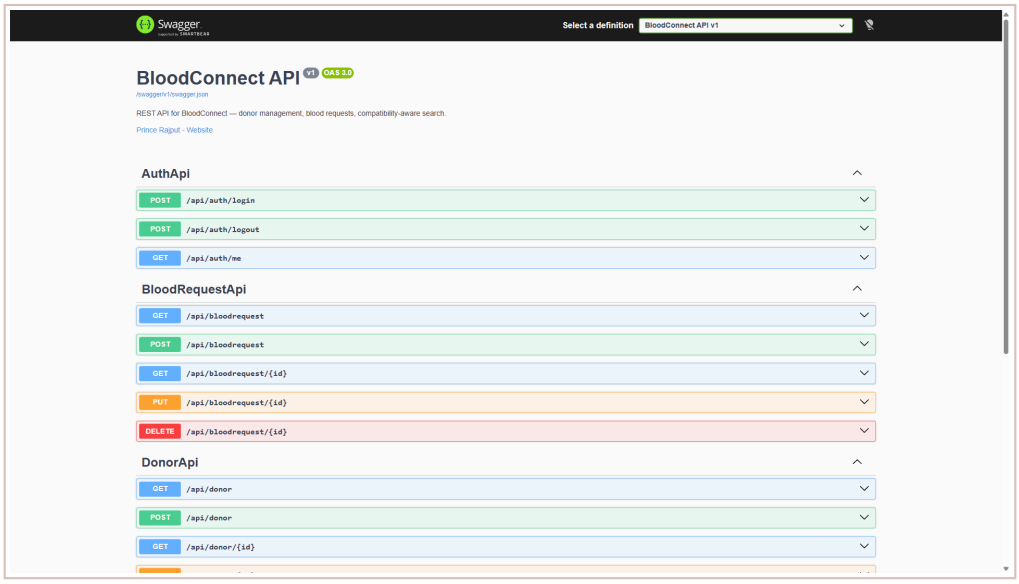


Figure 8.2: *Real-Time SignalR Notification Bell, Bouncing Unread Badge, and Pledge Workflow view showing masked contact defaults and authorized reveal on acceptance.*

9. PHASE 4: REST API, Cookie Authentication & Live Deployment

PHASE 4 ALIGNMENT — API, REAL-TIME SIGNALR & PRODUCTION DEPLOYMENT

Objective: Expose a secure REST API with Cookie auth, SignalR real-time WebSocket notifications, Swagger UI, and live HTTPS deployment.

Key Deliverables: DonorApiController, BloodRequestApiController, AuthApiController, NotificationHub, 28 Postman tests, and MonsterASP deployment.

Phase 4 delivers enterprise interoperability, real-time notifications, and live cloud deployment. REST API controllers under **/api/donor**, **/api/bloodrequest**, and **/api/auth** provide full CRUD, compatibility-aware search, and JSON authentication issuing the **.AspNetCore.Identity.Application** cookie. Live push notifications are delivered over WebSockets via SignalR **NotificationHub (/hubs/notifications)** with a two-tone Web Audio API chime (880Hz + 1174Hz).

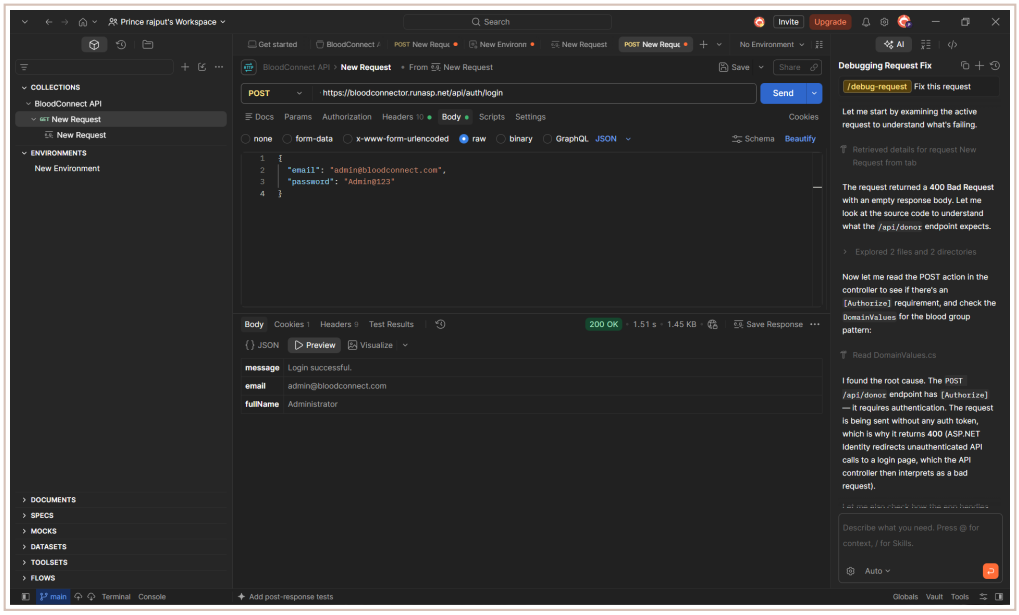


Figure 9.1: *Postman verification: POST /api/auth/login authenticating via JSON and issuing a valid .AspNetCore.Identity.Application session cookie (200 OK).*

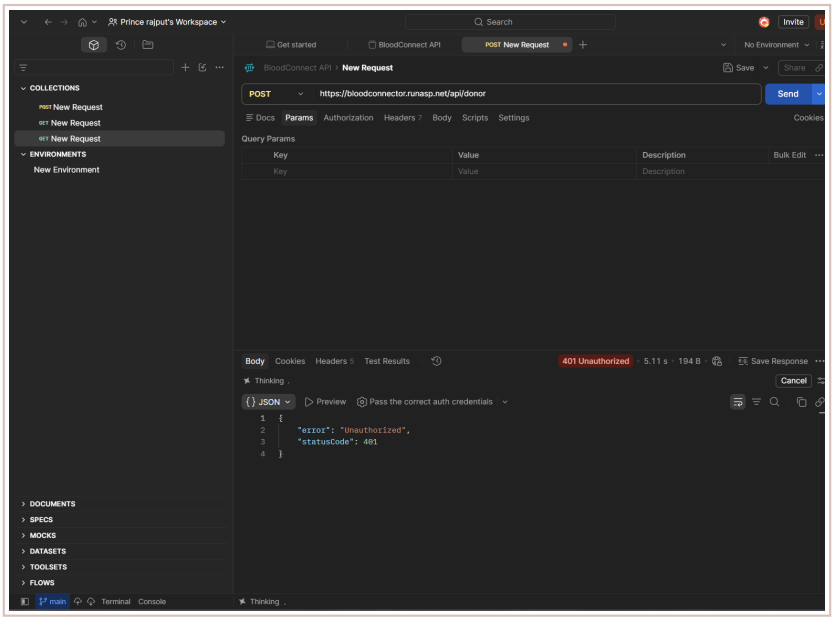


Figure 9.2: *Postman verification: Unauthenticated POST /api/donor request returning HTTP 401 Unauthorized JSON without HTML redirect.*

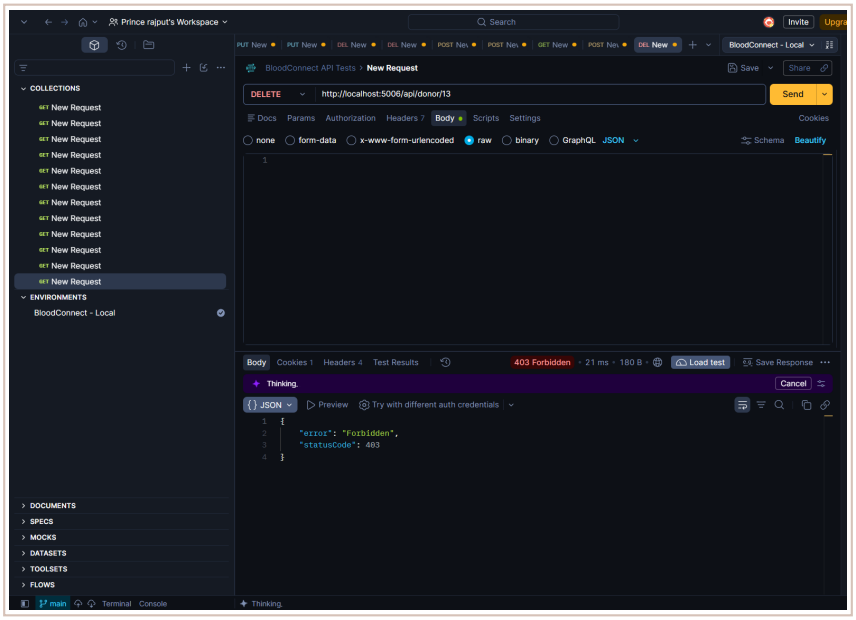


Figure 9.3: *Postman verification: Cross-user IDOR DELETE /api/donor/{id} attempt blocked by server-side ownership check (HTTP 403 Forbidden).*

10. Testing Verification Matrix & Challenges

SYSTEM VERIFICATION & RETROSPECTIVE

Verification: 28 Postman API tests and 11 critical test cases verified across security invariants, CRUD workflows, API responses, and deployment stability.

Deployment URL: https://bloodconnector.runasp.net (MonsterASP Windows Hosting with Let's Encrypt SSL).

Test Case Description	Expected Behavior / Requirement	Status
IDOR Cross-User Access	Attempting to edit or delete another user's donor profile returns 403 Forbidden	**PASS ✔**
RBC Compatibility Search	Searching recipient A+ returns only compatible O-, O+, A-, A+ donors in O(1)	**PASS ✔**
Age & State Validation	Donor age under 18 or over 65 or invalid state 'Wakanda' rejected with HTTP 400	**PASS ✔**
API Missing Auth Cookie	Calling POST /api/donor without authentication returns HTTP 401 (no redirect)	**PASS ✔**
Duplicate Donor Conflict	Submitting a second donor profile for the same user returns HTTP 409 Conflict	**PASS ✔**
Account Lockout	5 failed login attempts trigger a 5-minute account lockout protection	**PASS ✔**
Banned User Eviction	Setting IsBanned = true immediately signs out user via OnValidatePrincipal	**PASS ✔**
Pledge Auto-Reject Rule	Accepting one DonationOffer auto-rejects competing offers & reveals phone	**PASS ✔**
Optimistic Concurrency	Simultaneous conflicting edits on Donor throw DbUpdateConcurrencyException	**PASS ✔**
XSS & SQLi Defense	Script tags HTML-encoded by Razor; SQLi payloads parameterized by EF Core	**PASS ✔**
Mobile Responsiveness	Bootstrap 5 layout & SignalR notifications adapt cleanly across viewports	**PASS ✔**

11. References & Bibliography

- **[1] Microsoft Docs:** ASP.NET Core 10 Overview, Entity Framework Core 10, and ASP.NET Core Identity documentation. https://learn.microsoft.com/
- **[2] ASP.NET Core SignalR:** Real-Time WebSocket Hub Architecture, Group Broadcasts, and Client Reconnection. https://learn.microsoft.com/en-us/aspnet/core/signalr/
- **[3] OWASP Foundation:** Top Ten Web Application Security Risks — A01: Broken Access Control (IDOR), A03: Injection (SQLi/XSS) & Authentication Best Practices. https://owasp.org/
- **[4] OpenAPI & RFC 9110:** Swashbuckle.AspNetCore Swagger UI Documentation and RFC 9110 ProblemDetails Error Standard. https://datatracker.ietf.org/doc/html/rfc9110
- **[5] Course Curriculum:** Full Stack Web Development Project using ASP.NET Core (DSC-M-BCA-471P), guided by Prof. Sejal Shah at Lokmanya College of Computer Applications (Gujarat University), Academic Year 2026–27.